
Trabalho de Banco de Dados - Brechó

1. Introdução

Este trabalho tem como objetivo modelar e implementar um sistema de banco de dados para um brechó. A proposta é criar uma estrutura de dados eficiente que permita gerenciar funcionários, clientes, roupas, numerações disponíveis e pedidos realizados. Além disso, serão demonstradas consultas úteis para o gerenciamento do brechó.

2. Requisitos do Sistema

O sistema deverá atender aos seguintes requisitos:

- **Armazenar informações dos funcionários:** nome, cargo e salário.
- **Cadastrar clientes:** nome, idade e gênero.
- **Manter um catálogo de roupas:** nome e preço.
- **Associar numerações disponíveis** a cada roupa.
- **Registrar pedidos** feitos por clientes.
- **Controlar os itens** presentes em cada pedido.

3. Diagrama Entidade-Relacionamento (DER)

Entidades:

- **Funcionários:** (ID_funcionario, Nome, Cargo, Salario)
- **Clientes:** (ID_cliente, Nome, Idade, Genero)
- **Roupas:** (ID_roupa, Nome, Preco)
- **Numeracoes_disp:** (ID_roupa, Numeracoes)
- **Pedidos:** (ID_pedido, Data, ID_cliente)
- **Itens_pedidos:** (ID_pedido, ID_roupa, Quantidade)

Relacionamentos:

- Cada **cliente** pode fazer vários **pedidos** (1:N).

- Cada **pedido** pode conter várias **roupas** (N:N com **Itens_pedidos**).
- Cada **roupa** pode ter várias **numerações disponíveis** (1:N).

4. Normalização (Comentada)

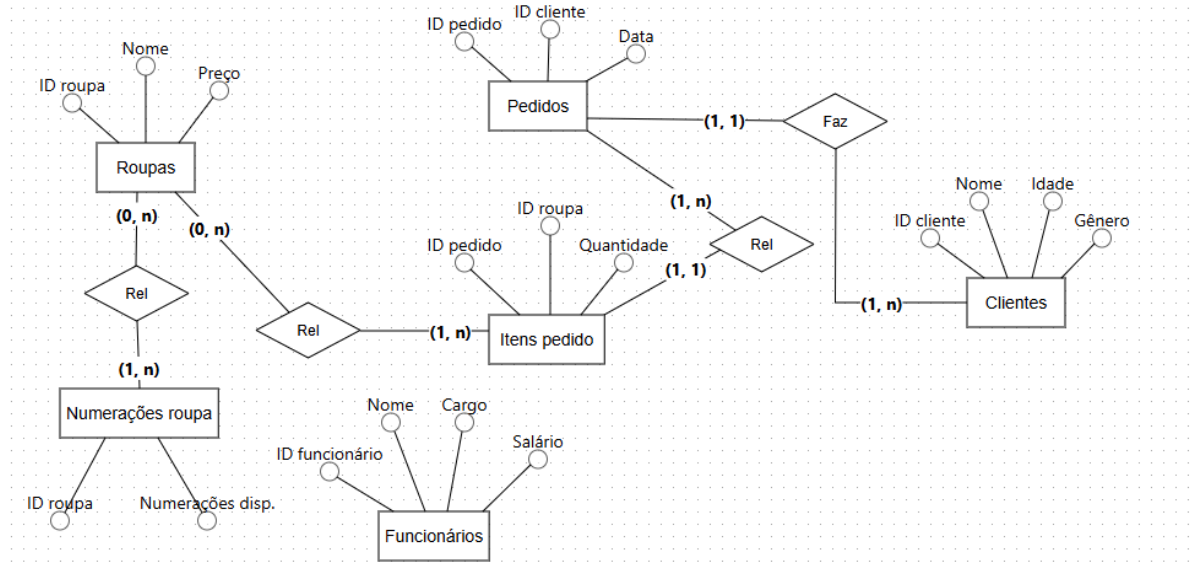
Todas as tabelas seguem até a **3FN (Terceira Forma Normal)**, conforme abaixo:

- **1FN (Primeira Forma Normal)**: Eliminamos grupos repetitivos e garantimos a atomicidade dos dados. Cada campo contém um valor único e indivisível.
- **2FN (Segunda Forma Normal)**: Garantimos que não haja dependência parcial. Ou seja, todos os atributos não-chave dependem da chave primária completa. Não podemos ter dependências parciais, ou seja, atributos que dependem de apenas uma parte da chave composta.
- **3FN (Terceira Forma Normal)**: Garantimos que não haja dependência transitiva. Ou seja, atributos não-chave não dependem de outros atributos não-chave. Isso elimina qualquer tipo de dependência indireta.

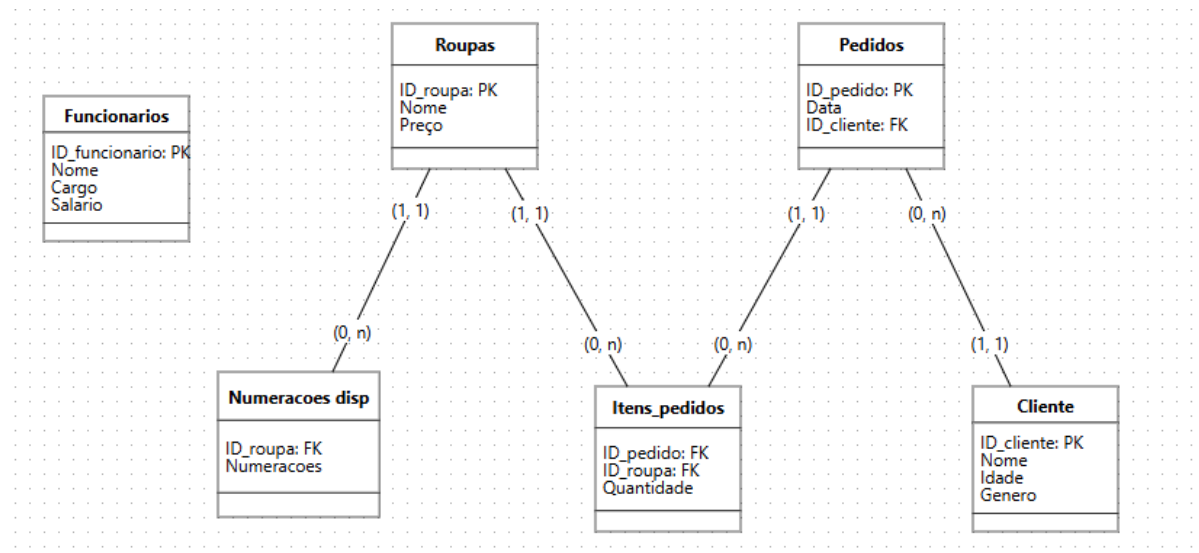
Exemplos:

- **Tabela 'Funcionarios'**: Todos os atributos estão diretamente ligados à chave primária **ID_funcionario**, garantindo que não haja dependências de outros atributos não-chave.
- **Tabela 'Itens_pedidos'**: Usa chave composta (**ID_pedido**, **ID_roupa**), evitando duplicações e mantendo a integridade relacional. Não há dependências parciais, pois os atributos dependem da chave completa.
- **Tabela 'Numeracoes_disp'**: Permite múltiplas numerações por roupa sem violar a **1FN**, pois cada linha armazena uma única numeração para cada roupa, mantendo os dados atômicos.

Modelagem Conceitual



Modelagem Lógica



Código das tabela MySQL

```
--CRIAÇÃO DAS TABELAS

CREATE TABLE Funcionarios (
  ID_funcionario INT AUTO_INCREMENT PRIMARY KEY,
```

```

    Nome VARCHAR(100),
    Cargo VARCHAR(50),
    Salario DECIMAL(10,2)
);

CREATE TABLE Cliente (
    ID_cliente INT AUTO_INCREMENT PRIMARY KEY,
    Nome VARCHAR(100),
    Idade INT,
    Genero VARCHAR(10)
);

CREATE TABLE Roupas (
    ID_roupa INT AUTO_INCREMENT PRIMARY KEY,
    Nome VARCHAR(100),
    Preco DECIMAL(10,2)
);

CREATE TABLE Numeracoes_disp (
    ID_roupa INT,
    Numeracoes VARCHAR(10),
    PRIMARY KEY (ID_roupa, Numeracoes),
    FOREIGN KEY (ID_roupa) REFERENCES Roupas(ID_roupa)
);

CREATE TABLE Pedidos (
    ID_pedido INT AUTO_INCREMENT PRIMARY KEY,
    Data DATE,
    ID_cliente INT,
    FOREIGN KEY (ID_cliente) REFERENCES Cliente(ID_cliente)
);

CREATE TABLE Itens_pedidos (
    ID_pedido INT,
    ID_roupa INT,
    Quantidade INT,
    PRIMARY KEY (ID_pedido, ID_roupa),
    FOREIGN KEY (ID_pedido) REFERENCES Pedidos(ID_pedido),
    FOREIGN KEY (ID_roupa) REFERENCES Roupas(ID_roupa)
);

--INSERÇÃO DE DADOS

```

```
INSERT INTO Funcionarios (Nome, Cargo, Salario) VALUES ('Erick
Gabriel', 'Gerente', 3500.00);
INSERT INTO Funcionarios (Nome, Cargo, Salario) VALUES ('João Pedro',
'Estoquista', 2000.00);
INSERT INTO Funcionarios (Nome, Cargo, Salario) VALUES ('João Gabriel',
'Caixa', 2000.00);
INSERT INTO Funcionarios (Nome, Cargo, Salario) VALUES ('Arthur
Barbosa', 'Faxineiro', 180.00);

INSERT INTO Cliente (Nome, Idade, Genero) VALUES ('Isaque Cristo', 16,
'Neutro');
INSERT INTO Cliente (Nome, Idade, Genero) VALUES ('William Freitas',
35, 'Masculino');
INSERT INTO Cliente (Nome, Idade, Genero) VALUES ('William Júnior', 19,
'Masculino');
INSERT INTO Cliente (Nome, Idade, Genero) VALUES ('Maria Eduarda', 16,
'Feminino');

INSERT INTO Roupas (Nome, Preço) VALUES ('Camiseta Apex Twin', 59.90);
INSERT INTO Roupas (Nome, Preço) VALUES ('Calça Ultra Baggy Jeans',
229.99);
INSERT INTO Roupas (Nome, Preço) VALUES ('Vestido Floral', 149.50);
INSERT INTO Roupas (Nome, Preço) VALUES ('Jaqueta Couro', 149.50);
INSERT INTO Roupas (Nome, Preço) VALUES ('Jorts Baggy', 129.90);
INSERT INTO Roupas (Nome, Preço) VALUES ('Calcinha Renda', 49.90);

INSERT INTO Numeracoes_disp VALUES (1, 'P');
INSERT INTO Numeracoes_disp VALUES (1, 'M');
INSERT INTO Numeracoes_disp VALUES (2, '40');
INSERT INTO Numeracoes_disp VALUES (2, '42');
INSERT INTO Numeracoes_disp VALUES (3, 'G');
INSERT INTO Numeracoes_disp VALUES (6, 'M');

INSERT INTO Pedidos (Data, ID_cliente) VALUES ('2025-07-01', 1);
INSERT INTO Pedidos (Data, ID_cliente) VALUES ('2025-07-02', 2);
INSERT INTO Pedidos (Data, ID_cliente) VALUES ('2025-07-03', 3);
INSERT INTO Pedidos (Data, ID_cliente) VALUES ('2025-07-04', 1);
INSERT INTO Pedidos (Data, ID_cliente) VALUES ('2025-07-05', 4);

INSERT INTO Itens_pedidos VALUES (1, 6, 1);
INSERT INTO Itens_pedidos VALUES (1, 3, 1);
INSERT INTO Itens_pedidos VALUES (2, 3, 1);
```

```

INSERT INTO Itens_pedidos VALUES (3, 4, 2);
INSERT INTO Itens_pedidos VALUES (4, 2, 1);

--Consulta

SELECT * FROM Cliente;

SELECT * FROM Cliente
  WHERE idade>18;

SELECT * FROM Pedidos
WHERE ID_cliente>2 and Data>'2025-07-01';

SELECT nome,Preco FROM Roupas
WHERE Preco>50;

SELECT nome,Preco FROM Roupas
ORDER By Preco DESC;

SELECT nome,Preco FROM Roupas
ORDER By Preco ASC;

SELECT Salario FROM Funcionarios
WHERE Salario>180;

--Junta as tabelas
SELECT Cliente.Nome AS cliente, Roupas.Nome AS produto, Pedidos.Data
FROM Pedidos
JOIN Cliente ON Pedidos.ID_cliente = Cliente.ID_cliente
JOIN Itens_pedidos ON Pedidos.ID_pedido = Itens_pedidos.ID_pedido
JOIN Roupas ON Itens_pedidos.ID_roupa = Roupas.ID_roupa;

--Estoque de roupas
SELECT COUNT(*) AS total_sem_estoque
FROM Roupas
LEFT JOIN Numeracoes_disp ON Roupas.ID_roupa = Numeracoes_disp.ID_roupa
WHERE Numeracoes_disp.ID_roupa IS NULL;

--Conta os numeros de clientes
SELECT COUNT(*) FROM Cliente

---Conta por condicao
SELECT COUNT(*) FROM Cliente WHERE Idade > 18;

```

```
--Conta o numero de pedidos dos clientes
SELECT ID_cliente, COUNT(*) AS total_pedidos
FROM Pedidos
GROUP BY ID_cliente;
```

```
--Conta quantos itens tem em cada pedido
SELECT ID_pedido, COUNT(*) AS total_itens
FROM Itens_pedidos
GROUP BY ID_pedido;
```

```
--Remover por condicao
DELETE FROM Cliente WHERE ID_cliente = 3;
```

```
--Remover tudo
DELETE FROM Funcionarios;
```